



United International University
Department of CSE

Course Code: CSE 4326

Course Name: Microprocessors and Microcontrollers
Laboratory

Experiment no : 01

Experiment Name: Basic Interfacing of Input and Output
Devices and Sensors with Arduino Uno

Submitted by (Group no.): 01

Name : Sourav Saha

Student ID: 0112330638

Section: A

Date of Performance: 11 July 2026

Date of Submission : 21 July 2026

Objective:

The objective of this experiment is to understand the basic interfacing of input and output devices with Arduino Uno. In this experiment, a push button is used to control an LED and a buzzer. An LDR sensor is used to detect changes in light intensity and control an LED according to the surrounding light condition. Finally, an ultrasonic sensor is interfaced with Arduino to measure the distance of an object. Through these tasks, I learned how Arduino reads digital and analog inputs, controls output devices, and processes sensor data.

Apparatus:

- Arduino Uno
- Push Button
- LED
- Buzzer
- LDR Sensor
- Ultrasonic Sensor
- Resistors
- Breadboard
- Jumper Wires
- Arduino IDE
- Proteus Software

Theory:

Arduino Uno is a microcontroller-based development board that can receive signals from different input devices and control output devices according to the uploaded program. It contains both digital and analog pins. Digital pins are mainly used to read or generate HIGH and LOW signals, while analog pins can read varying voltage levels from sensors.

A push button is a simple digital input device. Its state changes when it is pressed or released. Arduino can read this state using a digital input pin and perform a specific operation, such as controlling an LED or a buzzer.

An LDR, or Light Dependent Resistor, is a sensor whose resistance changes with the intensity of light. Its resistance becomes lower when the surrounding light intensity increases and becomes higher when the light intensity decreases. By using an LDR in a

voltage divider circuit, Arduino can read the change in voltage through an analog input pin and control an output device according to the measured light level.

An ultrasonic sensor measures distance by transmitting ultrasonic sound waves and receiving the reflected echo. The sensor sends a short ultrasonic pulse toward an object. After hitting the object, the sound wave returns to the sensor. Arduino measures the travel time of the sound wave and calculates the distance.

The distance is calculated using:

$$\text{Distance} = \frac{\text{Time} \times \text{Speed of Sound}}{2}$$

The result is divided by two because the measured time includes both the forward journey of the sound wave toward the object and its return journey to the sensor.

Solution of the mentioned problems:

Problem 1: Use a Push Button to Toggle an LED

Solution:

In this task, I interfaced a push button and an LED with Arduino Uno. The push button was used as the input device, while the LED was used as the output device.

The main purpose of the program was to change the current state of the LED whenever the button was pressed. If the LED was OFF, pressing the button turned it ON. If the LED was already ON, pressing the button turned it OFF.

To implement this operation, the program continuously read the state of the push button. I also stored the current state of the LED in a variable. A change in the button state was detected so that holding the button did not repeatedly toggle the LED.

After completing the circuit connections, I uploaded the program to the Arduino and tested the circuit. The LED successfully changed its state after every button press.

Code for Problem 1:

```
void setup()
{
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
  pinMode(13, OUTPUT);
}
void loop()
```

```

{
  int sensorVal = digitalRead(2);
  Serial.println(sensorVal);
  if (sensorVal == LOW)
  {
    digitalWrite(13, HIGH);
  }
  else
  {
    digitalWrite(13, LOW);
  }
}

```

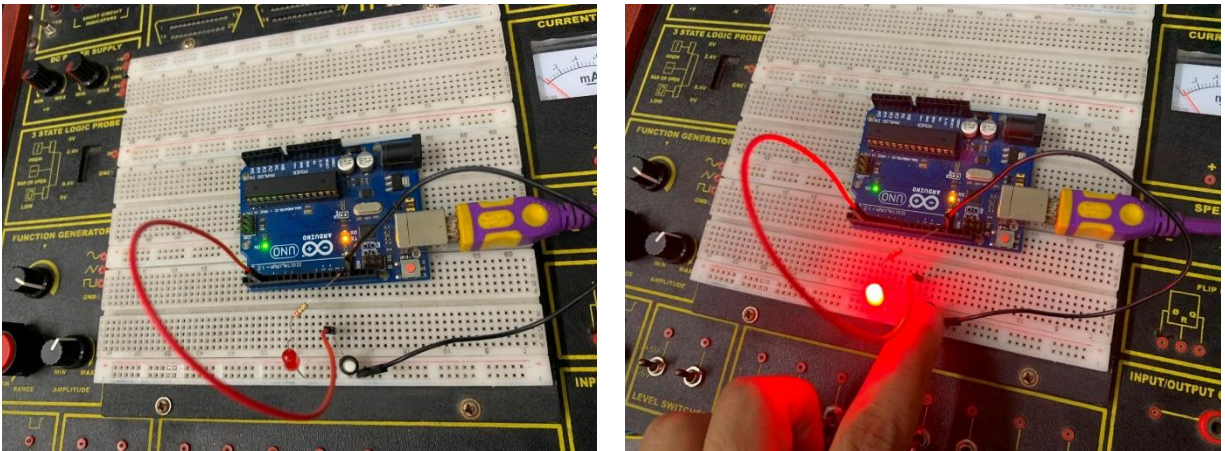


Figure: Problem 1

Problem 2: Use a Push Button to Turn ON a Buzzer

Solution:

In this task, I interfaced a push button and a buzzer with Arduino Uno. The push button worked as the input device, and the buzzer worked as the output device.

The program continuously monitored the state of the push button through a digital input pin. When the button was pressed, Arduino activated the output pin connected to the buzzer. As a result, the buzzer started producing sound.

When the push button was released, the output signal was changed to the inactive state and the buzzer stopped producing sound.

After completing the connections and uploading the program, I tested the circuit by pressing and releasing the push button. The buzzer produced sound only while the required button condition was detected.

Code for Problem 2:

```
const int buzzerPin = 8;  
void setup() {  
  pinMode(buzzerPin, OUTPUT);  
}  
void loop() {  
  digitalWrite(buzzerPin, HIGH);  
  delay(1000);  
  digitalWrite(buzzerPin, LOW);  
  delay(1000);  
}
```

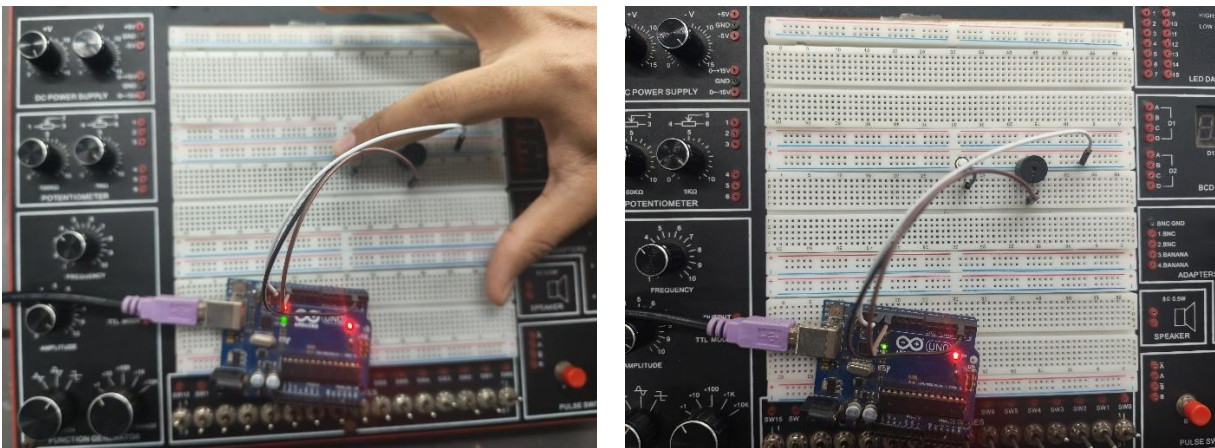


Figure: Problem 2

Problem 3: Interfacing an LDR Sensor with Arduino to Control an LED

Solution:

In this task, I interfaced an LDR sensor with Arduino Uno to control an LED according to the surrounding light intensity.

The LDR was connected in a voltage divider circuit, and the output of the voltage divider was connected to an analog input pin of Arduino. The LED was connected to a digital output pin.

Arduino continuously read the analog value produced by the LDR circuit. A threshold value was used in the program to decide whether the surrounding light intensity was low or high.

When the light intensity was low, the measured sensor value crossed the selected condition and Arduino turned the LED ON. When the light intensity was high, Arduino turned the LED OFF.

I tested the circuit under both low-light and high-light conditions. The LED successfully turned ON when the surrounding light intensity was low and turned OFF when the light intensity was high.

Code for Problem 3:

```
const int LDR_PIN = A0;
const int LED_PIN = 11;
int ldrValue = 0;
int threshold = 500;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  ldrValue = analogRead(LDR_PIN);
  Serial.print("LDR Value: ");
  Serial.println(ldrValue);
  if (ldrValue < threshold) {
    digitalWrite(LED_PIN, HIGH);
  }
  else {
    digitalWrite(LED_PIN, LOW);
  }
  delay(500);
}
```

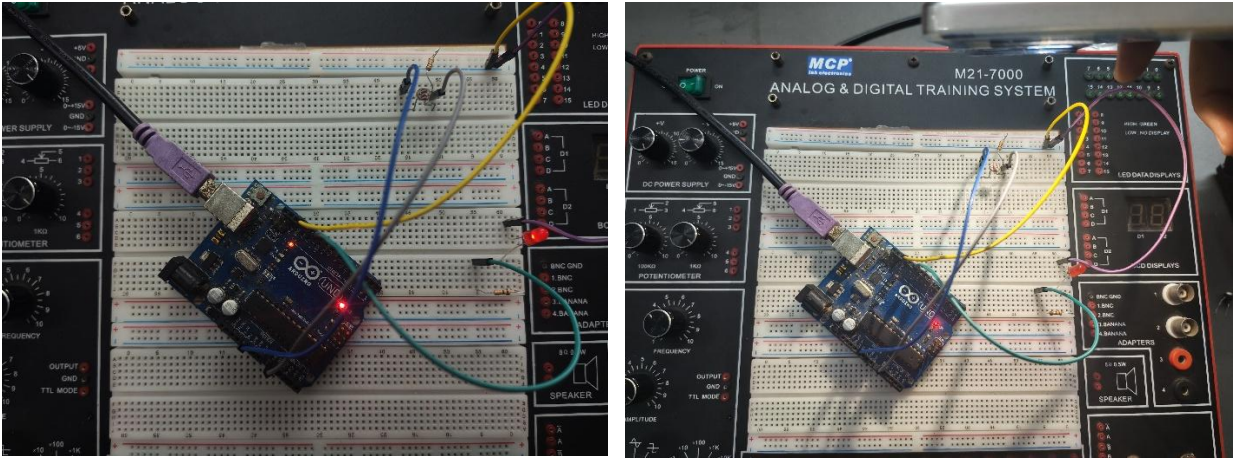


Figure: Problem 3

Problem 4: Measuring the Distance of an Object Using a SONAR Sensor

Solution:

In this task, I interfaced an ultrasonic sensor with Arduino Uno to measure the distance of an object.

The ultrasonic sensor uses a trigger pin to transmit an ultrasonic pulse and an echo pin to receive the reflected pulse. The trigger and echo pins were connected to the digital pins of Arduino.

The program first generated a short pulse through the trigger pin. The ultrasonic sensor transmitted a sound wave toward the object. After hitting the object, the sound wave returned to the sensor and produced a signal at the echo pin.

Arduino measured the duration of the echo pulse. The measured time was then used to calculate the distance of the object. Since the sound wave travels from the sensor to the object and returns to the sensor, the calculated distance was divided by two.

The measured distance was converted into meters and displayed as the output of the experiment.

I tested the circuit by changing the position of the object. The measured distance changed according to the position of the object, and the distance value was successfully displayed in meters.

Code for Problem 4:

```
const int trigPin = 9;
const int echoPin = 10;
void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  digitalWrite(trigPin, LOW);
}
void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(5);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  unsigned long duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) {
    Serial.println("ERROR: No Echo Received");
  } else {
    float distanceMeter = duration * 0.000343 / 2.0;
    Serial.print("Duration: ");
    Serial.print(duration);
    Serial.print(" us | Distance: ");
    Serial.print(distanceMeter, 3);
    Serial.println(" meter");
  }
  delay(500);
}
```

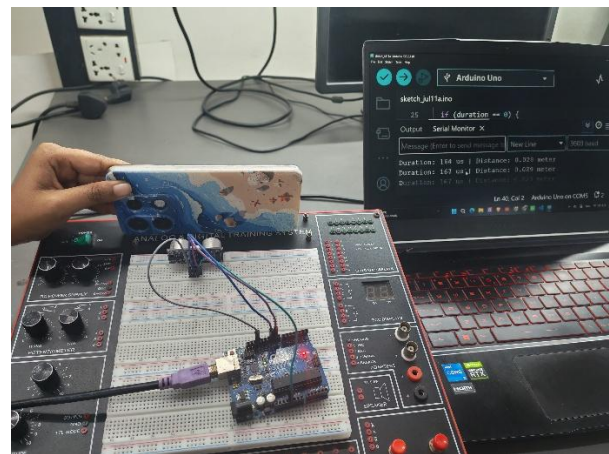
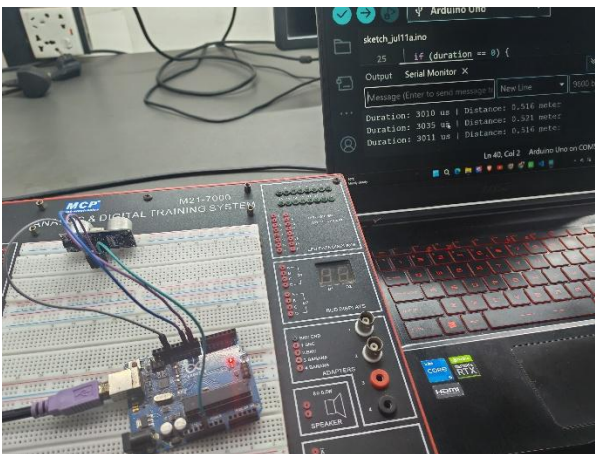


Figure: Problem 4

Discussion:

In this experiment, I completed four Arduino interfacing tasks using different input and output devices. First, I used a push button to toggle the state of an LED. Then, I controlled a buzzer using another push-button-based circuit. These two tasks helped me understand how Arduino reads digital input signals and controls digital output devices.

In the third task, I used an LDR sensor to detect changes in light intensity. The analog value obtained from the sensor was compared with a threshold value to control an LED. This task helped me understand the use of analog input and conditional control in Arduino.

Finally, I used an ultrasonic sensor to measure the distance of an object. The distance was calculated from the travel time of the ultrasonic pulse and displayed in meters. From this task, I learned the basic working principle of ultrasonic distance measurement and how sensor data can be processed using Arduino.

All four circuits were tested with the required conditions, and the expected outputs were obtained.